

ТЕХНИЧЕСКА ДОКУМЕНТАЦИЯ

„Chess App“ — веб приложение за шахмат в реално време

Изготвено от Екип 8:

Александра	Недкова	Фн:1MI0700210
Васил	Палигоров	Фн:6MI0700192
Ивалин	Генов	Фн:9MI0700224
Мартин	Монков	Фн:1MI0700223
Яна	Христева	Фн:5MI0700216



1. Предназначение на системата

„Chess App“ е клиент-сървър уеб приложение за игра на шахмат между двама регистрирани потребители в реално време. За разлика от приложения от типа на Lichess или Chess.com, целта тук не е изграждане на платформа с изкуствен интелект, турнирни таблици или времеви контрол на партиите — фокусът е върху правилно реализирана архитектура клиент-сървър с авторитетна сървърна логика и комуникация в реално време чрез WebSockets. Системата поддържа пълните правила на шахмата и всички начини за завършване на партия.

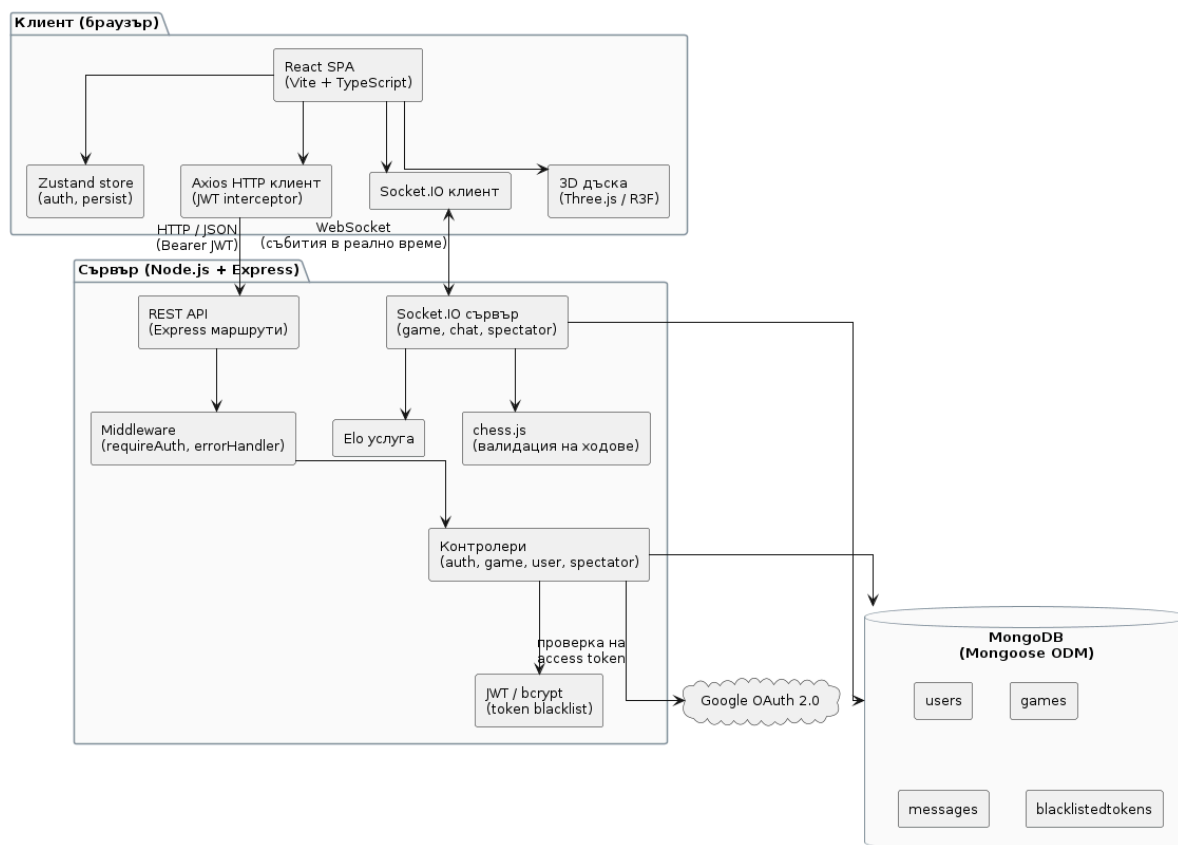
Основни функционалности на системата:

- регистрация и вход с парола или Google акаунт (OAuth 2.0);
- лоби с чакащи партии, обновявано в реално време; създаване и присъединяване към партия;
- игра в реално време със сървърна валидация на всеки ход; предложение за реми и предаване;
- чат между играчите в игра;
- режим „зрител“ чрез споделяне на линк;
- рейтингова система Elo, класация, профили със статистика, история на ходовете и визуализация на игрите, редактиране на собствен профил;
- 3D визуализация на дъската чрез Three.js / React Three Fiber.

2. Архитектура

Проектът е разделен на три модула: client/ (React + TypeScript SPA, изграждан с Vite), server/ (Node.js + Express + TypeScript) и shared/ (общи TypeScript типове и имена на Socket.IO събитията, използвани от двете страни — това гарантира типова съгласуваност на протокола). Комуникацията е по два канала: REST API (HTTP/JSON) за автентикация и управление на партии и Socket.IO (WebSocket) за събитията в реално време — ходове, чат, край на партия, обновяване на лобито. Данните се съхраняват в MongoDB чрез Mongoose.

Игровата логика е сървърна. Всеки ход се валидира със сървърно копие на партията (chess.js), възстановявано от съхранения PGN запис; клиентът само изпраща намерение за ход и визуализира потвърденото от сървъра състояние (FEN).



Фигура 1. Компонентна диаграма на системата

Сървърът следва структурата „маршрути → middleware (requireAuth, errorHandler) → контролери → Mongoose модели“, допълнена със Socket.IO слой (игра, чат, зрители) и услуги (Elo, черен списък с токени). Клиентът използва React Router със защитени маршрути, Zustand (persist) за сесията, Axios с interceptor-и за автоматично прикачане на JWT и изход при 401, и singleton Socket.IO връзка с токена в handshake. Игровата логика на клиента е капсулирана в кука useGame.

3. Използвани технологии

Frontend

Технология	Версия	Предназначение
React	19	UI рендериране и компонентна архитектура
TypeScript	5.x	Статично типизиран JavaScript
Vite	6	Build tool и dev server (port 5173)
Zustand	5	Глобален state container с persist middleware за localStorage

Технология	Версия	Предназначение
Socket.IO client	4	WebSocket-базирана комуникация със сървъра
react-chessboard	5	Готов компонент за визуализация на шахматната дъска
chess.js	1.4	Локална валидация на ходовете (UX превю, hint markers)
react-router-dom	7	Клиентско рутиране (SPA)
axios	latest	HTTP клиент за REST API call-ове
@react-oauth/google	latest	Google OAuth интеграция за алтернативно влизане
framer-motion	latest	Анимации на UI преходи
@react-three/fiber + drei + three	latest	По избор: 3D рендериране на дъската

Backend

Технология	Версия	Предназначение
Node.js	20+	JavaScript runtime
Express	4	HTTP сървър и routing layer
TypeScript	5.x	Статично типизиран Node.js код
Socket.IO server	4	WebSocket сървър за реално време
Mongoose	8	MongoDB ODM с поддръжка на schemas и индекси
chess.js	latest	Авторитетна валидация на ходове и краен резултат
bcrypt	5	Хеширане на пароли (12 salt rounds)
jsonwebtoken	9	JWT генериране и верификация
google-auth-library	latest	Верификация на Google ID токени
uuid	latest	Генериране на spectator токени

База данни

Технология	Версия	Предназначение
MongoDB	6+	Документна база за потребители, партии, съобщения и blacklist на токени

Обосновка на ключовите избори

- **MongoDB вместо SQL** — данните за партията (PGN, FEN, draw offer) са естествено документно-ориентирани и не изискват сложни JOIN заявки. Полу-структурираните полета (endReason, drawOffer.from) се поддържат тривиално.
- **Socket.IO вместо vanilla WebSocket** — Socket.IO предоставя готови абстракции като rooms, auto-reconnect и authentication middleware, които ускоряват разработката.
- **chess.js и в клиента, и в сървъра** — клиентът ползва chess.js за моментална UX обратна връзка (legal-move highlight, locally-rendered preview), но сървърът е винаги авторитетен. Това решение премахва дъбъл-логика и поддържа консистентност.
- **Zustand вместо Redux** — Zustand има минимален boilerplate и поддръжка на persist middleware, който запазва автентикационното състояние в localStorage без допълнителен код.
- **JWT в HTTP header + handshake.auth.token за socket** — единен механизъм за идентификация по двата канала, без cookies или сесии в базата.

4. Модел на данните

Данните са в четири MongoDB колекции с Mongoose схеми, уникални и TTL индекси; връзките са чрез референции (ObjectId) и populate.

Колекция	Основни полета
users	username и email (уникални), passwordHash (bcrypt, select: false), googleId, elo (начало 1200, индексирани), peakElo, wins/losses/draws, createdAt
games	whitePlayer и blackPlayer (реф. към User), status ('waiting' 'active' 'finished'), result, winner, fen (текуща позиция), pgn (пълнен запис), spectatorToken (UUID за зрители), drawOffer, endReason, createdAt/finishedAt
messages	gameId (индексирани), userId, username, text (до 200 символа), createdAt
blacklistedtokens	tokenHash (SHA-256 на инвалидиран JWT), expiresAt с TTL индекс — записите се изтриват автоматично след изтичане на токена

Жизненият цикъл на партията: waiting (създадена, видима в лобито) → active (втори играч се е присъединил) → finished (с result и endReason). Чакаща партия може да бъде отказана от създателя ѝ.

5. REST API

Единен формат на отговорите: { success: true, data } при успех, { success: false, message } при грешка (400/401/404/409). Защитените крайни точки изискват header Authorization: Bearer <token>.

Метод	Път	Защита	Описание
POST	/api/auth/register	не	Регистрация (username, email, password); връща потребител и JWT
POST	/api/auth/login	не	Вход с имейл/потребителско име и парола
POST	/api/auth/google	не	Вход/регистрация чрез Google access token
POST	/api/auth/logout	да	Изход; токенът влиза в черния списък, сокетите се прекъсват
GET	/api/auth/me	да	Данни на текущия потребител
POST	/api/games	да	Създаване на партия (създателят е белите)
GET	/api/games/waiting	да	Чакащи партии за лобито
GET	/api/games/me/current	да	Активните/чакащите партии на потребителя
GET	/api/games/spectate/:token	не	Данни за партия по зрителски токен
GET	/api/games/:id	да	Пълни данни за партия
POST	/api/games/:id/join	да	Присъединяване към чакаща партия
DELETE	/api/games/:id	да	Отказ от чакаща партия (само създателят)
GET	/api/users/leaderboard	да	Класация по Elo
GET	/api/users/:id	да	Публичен профил със статистика
GET	/api/users/:id/games	да	Завършени партии на потребителя

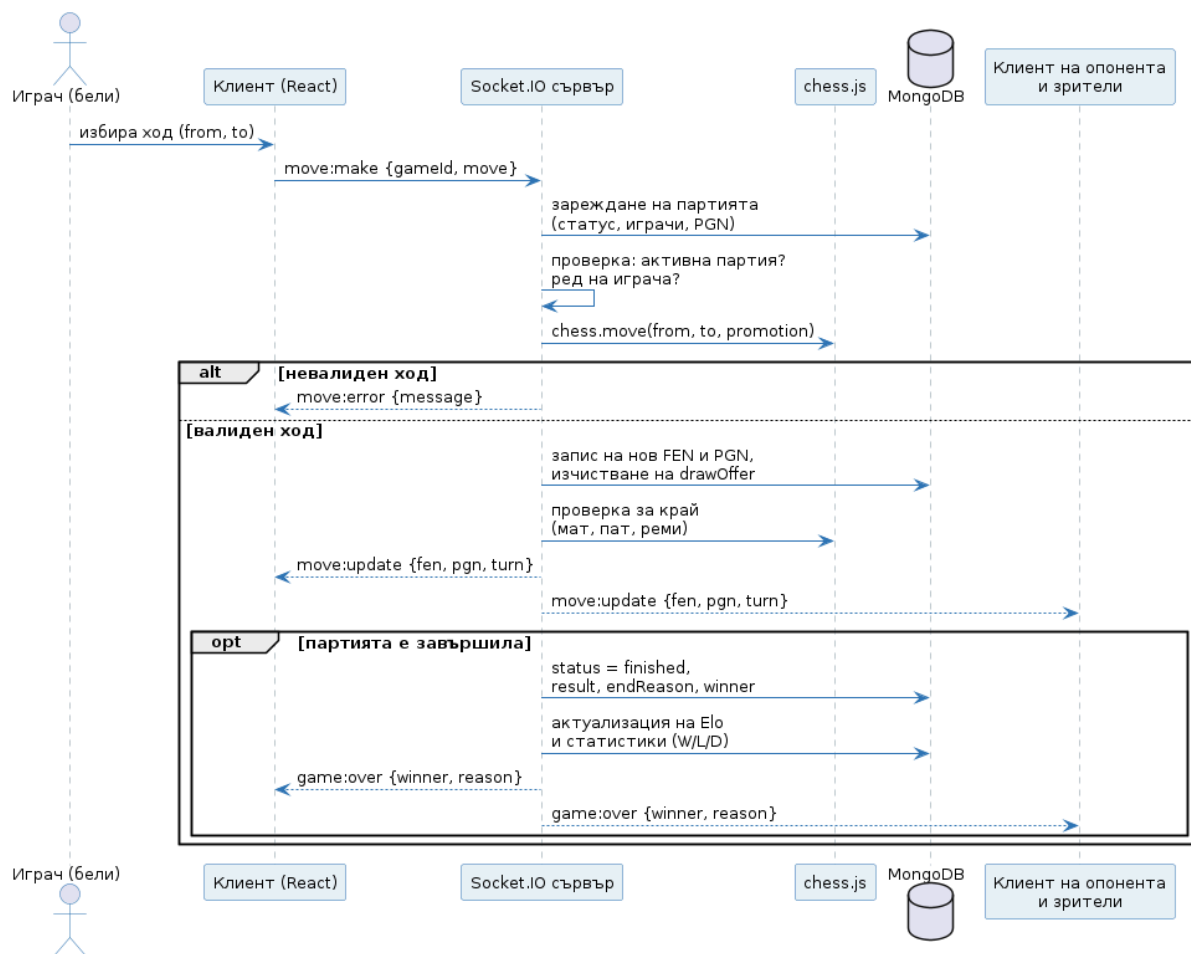
6. Комуникация в реално време (Socket.IO)

При свързване клиентът подава JWT в handshake данните; middleware проверява токена и черния списък и включва връзката в персонална стая user:<userId>. Комуникацията е организирана в стаи game:<gameId> (играчи) и spectate:<token> (зрители) — излъчванията за ходове и край на партия достигат и двете, но зрителите нямат права да изпращат игрови събития.

Посока	Събитие	Описание
клиент → сървър	game:join / game:leave	Влизане/излизане от стаята на партията; при влизане се изпраща историята на чата
клиент → сървър	move:make	Заявка за ход {gameId, move}; валидира се изцяло на сървъра

Посока	Събитие	Описание
клиент → сървър	game:resign	Предаване на партията
клиент → сървър	draw:offer / draw:accept / draw:decline	Предложение, приемане и отхвърляне на реми
клиент → сървър	chat:send	Чат съобщение (до 200 символа)
клиент → сървър	spectator:join / spectator:leave	Зрителски достъп по токен (без автентикация)
сървър → клиент	game:start	Партията започва (изпраща се до двамата играчи)
сървър → клиент	move:update	Потвърден ход {fen, pgn, turn}; до играчите и зрителите
сървър → клиент	move:error	Отказан ход (не е ваш ред / невалиден ход / неактивна партия)
сървър → клиент	game:over	Край на партията {winner, reason}
сървър → клиент	draw:offer / draw:declined	Получено/отхвърлено предложение за реми
сървър → клиент	chat:receive / chat:history	Ново съобщение / последните 50 съобщения
сървър → клиент	lobby:changed	Сигнал за промяна в лобито

Обработка на ход: сървърът проверява, че партията е активна, подателят е играч в нея и е на ход, и че ходът е легален (chess.js). При успех записва новите FEN/PGN, анулира активно предложение за реми, проверява за край на партията и излъчва обновлението. При завършена партия атомарно се актуализират Elo рейтингите, заедно с peakElo и броячите победи/загуби/реми.



Фигура 2. Диаграма на последователността при изпълнение на ход

7. Сигурност

- Паролите се съхраняват само като bcrypt хешове (12 salt rounds); полето е изключено от заявките по подразбиране.
- Сесии чрез JWT; при изход токенът влиза в черен списък (SHA-256 хеш с TTL индекс), проверяван при всяка HTTP заявка и Socket.IO връзка, а активните сокети на потребителя се прекъсват.
- Google токените се проверяват сървърно (google-auth-library) и се изисква потвърден имейл.
- Сървърна валидация на всички входни данни (формат на потребителско име, имейл, дължина на парола, ObjectId проверки) и контрол на достъпа на ниво операция: игрови действия — само от играч в партията, отказ — само от създателя, зрителите са само за четене.
- Игровата цялост е гарантирана от сървърната валидация на ходовете; зрителският достъп ползва непредвидим UUID токен, отделен от идентификатора на партията.

8. Инсталиране и стартиране

Изисквания: Node.js 18+, MongoDB (локална или Atlas). Инсталиране и стартиране в режим на разработка:

```
git clone https://github.com/VasilPaligorov/web-project-2026_chess-app.git  
cd web-project-2026_chess-app && npm run install:all && npm run dev
```

Сървърът работи на <http://localhost:3000>, клиентът — на <http://localhost:5173>.
Конфигурация чрез .env файлове: за сървъра — PORT, MONGO_URI, JWT_SECRET (задължителен), JWT_EXPIRES_IN, GOOGLE_CLIENT_ID; за клиента — VITE_API_URL и VITE_GOOGLE_CLIENT_ID.